Migrating Legacy Software to Microservices Architecture

Justas Kazanavičius
Vilnius Gediminas Technical University
Vilnius, Lithuania
J.Kazanavicius@gmail.com

Dalius Mažeika
Vilnius Gediminas Technical University
Vilnius, Lithuania
dalius.mazeika@vgtu.lt

***Abstract*—As a legacy monolithic software become too large to deal with, many enterprises are drawn to breaking them down into the microservices architectural style. Migrating a monolithic architecture to an ecosystem of microservices is an epic journey. Each legacy monolithic application is unique and migration to microservices creates different challenges. This article elaborates on challenges and technics of legacy software migration from monolithic architecture to microservice architecture. Different migration methods and techniques are reviewed, and their benefits and drawbacks are analyzed. Microservices is a relatively new architectural style and legacy monoliths are very different in many aspects. Existing migration techniques and methods do not cover every use case.**

Keywords—microservice, architecture, decomposition

I. Introduction

Monolithic architecture is the traditional way for software development when all functions are encapsulated into one single application. Monolithic software is designed to be self-contained. This type of architecture is tightly-coupled, and it means that if one of the components is not present then it will be not executed or compiled. Monolith architecture has owns benefits and drawbacks. The following benefits of Monolithic Architecture can be mentioned: Fewer Cross-cutting Concerns – it is simpler to hook up components to cross-cutting concerns when everything is running through the same application, Less Operational Overhead – only one application needs to be set up, Less complex to deploy – only one application needs to be deployed. Drawbacks of Monolithic Architecture are as follows: Coupled – is very difficult to make changes when monolith becomes very complex, Continuous deployment – the entire application should be deployed on each update, Scalability – difficult to scale when different modules have conflicting resource requirements, Reliability – bug in any component can potentially bring down the entire application.

Usually, legacy applications are always growing in size and complexity, therefore, it led to monstrous monolithic software after a few years of development and disadvantages of monolithic architecture become outweigh its advantages [28]. Fixing bugs and adding new features to such application is a very complex and time-consuming operation. Scalability usually is not possible or require a lot of work to do. Once this happens organizations start looking for a new architectural solution.

II. Microservice Architecture

According to Martin Fowler, the software developer who has written extensively about this topic, a microservice architecture consists of suites of independently deployable services organized around business capability, automated deployment, intelligence in the endpoints, and decentralized control of languages and data [12].

Microservices architecture (Figure 1) is an alternative way to overcome the challenges of monolithic architecture by decomposing the monolith into a set of small services and making them communicate with each other through lightweight mechanism like REST API or message bus. This approach allows to build and maintain application in simpler way when a solution is broken down into smaller pieces that work seamlessly together.

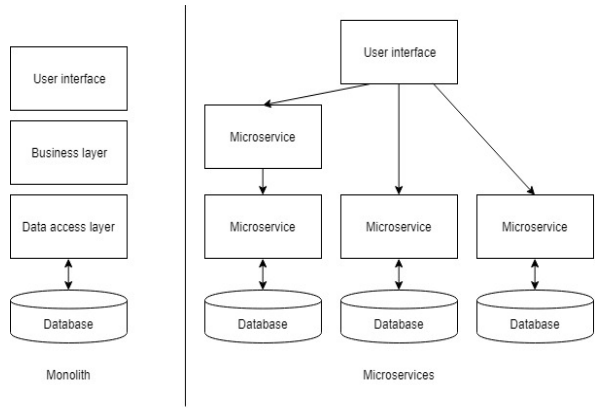

Fig. 1. Monolithic and microservices architectures.

Splitting application into distinct independent microservices allows managed them by individual teams within software development organization and work them independently. Usually, teams developing microservices are organized around business capabilities, but not technical capabilities. Each new requirement should be addressed by only one microservice to retain independent development [13].

The microservices architecture is not a panacea and there are benefits and drawbacks of using them [29]. Benefits of Microservices Architecture are as follows: Agility (Deployability) – microservices can be deployed independently and there is no need to restart an entire application, Reliability – a microservice fault affects that microservice alone and its consumers, Scalability – each microservice can be scaled independently using pools, clusters, grids, Cloudability – the deployment characteristics make microservices a great match for the elasticity of the cloud, Modifiability – each microservice is incapsulated therefore it is more flexibility to use new frameworks, libraries, data sources, and other resources. Management of the microservices based application

978-1-7281-2499-5/19/$31.00 ©2019 IEEE

Authorized licensed use limited to: Auckland University of Technology. Downloaded on August 21,2021 at 21:26:17 UTC from IEEE Xplore. Restrictions apply.

development is divided across the teams that are smaller and work more independently. Microservices architecture allows achieving better alignment of developers with business users since microservice architecture is organized around business capabilities and developers can easier understand user perspective and to create microservices that are better aligned with the business needs. As for drawbacks of Microservices Architecture it can be mentioned: Complexity – microservices architecture adds complexity to the project just by the fact that a microservices application is a distributed system, Performance – services more likely need to communicate over the network, whereas services within the monolith may benefit from local calls.

A. Monolith or Microservices

The monolith is not dead and microservices are not the best-suited for every context. It is a bad idea to dive into microservices simply because of their explosion in growth. It is necessary for developers and architects to closely evaluate whether the decomposition of an existing monolith is the right path and whether the microservices itself is the right destination [2].

A Monolithic architecture better suits a simple, lightweight application. The Microservices architecture solution is the better choice for complex, evolving applications. Monolith should be modernizing to microservices when:

- Monolith became too big and complex to maintain or extend.
- Modularity and decentralization are an important aspect.
- Preference for long term benefits in comparison to those in the short term.

III. Infrastructure Requirements for Microservices

An environment which supports microservices fundamentally needs a set of baseline requirements to ensure some level of sanity. If an organization is going to run it must be willing to bear the overhead of starting and supporting them. The overhead will not be insignificant. It will take time and money to do microservices well [14].

Each organization must have an internal group responsible for infrastructure which will be provided for development and operation teams to use microservices. This group must consist of best developers of organization or may even consultants, not from the organization.

There is no one rule for how to set up infrastructure for microservices. Each area below describes functionality which should be implemented in infrastructure.

A. Continuous Integration/Continuous Delivery

An organization should decide how microservices should be built, test and deploy. These operations should be done automatically [11, 15]. Nowadays there are plenty of build systems which provides pipeline functionality [5, 9]. Group responsible for microservice infrastructure should decide strategy on how to do it and choose tools for it:

- Source control – where and how should be stored and maintained source code.
- Build tool – how microservice should be built.
- Tests tool – how tests should be runned.
- Deploy tool – how microservice should be deployed.

B. Virtual Machines/Containers and Cloud

Another important decision is what technology to use for the execution environment. Many enterprises with existing applications running on a stable virtual machine infrastructure are choosing to take a 'toe in the water' approach. By deploying containers on virtual machines, they get the benefits of mature monitoring and isolation with more rapid DevOps processes. Compared to containers running on bare metal, they do give up some performance, scalability, and cost. But it's certainly a valid way to transition [16].

Microservice architectures are a natural fit for cloud native applications A cloud native application is simply defined as an application built from the ground up for cloud computing architectures. This simply means that our application is cloud native if we design it as if it is expected to be deployed on a distributed, and scalable infrastructure [3, 4, 10].

C. Monitoring

Monitoring is a critical piece of the infrastructure of microservices. Leveraging below listed five principles which allow an organization to establish more effective monitoring. These principles will allow organization to address both the technological changes associated with microservices, in addition to the organizational changes related to them [17].

- Monitor containers and what's inside them.
- Alert on service performance, but not on container performance.
- Monitor services that are elastic and have multi-location.
- Monitor APIs.
- Map your monitoring to your organizational structure.

D. Logging

It is vital to log information about microservices in production. To do it efficiently a logging service should be centralized and have strong visualizer. Nowadays there are plenty of tools which provides such functionality. Below listed best practice for logging microservices [18, 19, 20]:

- Correlate Requests with a Unique ID.
- Include a Unique ID in the Response.
- Structure Your Log Data.
- Add Context to Every Request.
- Write Logs to Local Storage.
- Log Useful and Meaningful Data to Avoid Regret.

Authorized licensed use limited to: Auckland University of Technology. Downloaded on August 21,2021 at 21:26:17 UTC from IEEE Xplore. Restrictions apply.

IV. Legacy Software Migration Methods

Migrating to microservices from monolithic legacy software cannot be changed in a fast way. It is important to know that there is a high overall cost associated with decomposing an existing system to microservices and it may take many iterations [1, 2]. Because enterprise legacy application is very broad term it is not possible to say that is only one good way to do migration from legacy monolith to microservices architecture [22, 23, 24]. Because microservices is a relatively new architectural style and no widely approved way of doing migration exist, different organizations use different migration patterns and techniques [26, 27].

A key challenge in this context is the extraction of microservices from existing legacy monolithic code bases [7, 25]. This chapter gives a review on different technics used to accomplish migration. In general, where are two strategies i.e. rebuilding and modernization (refactoring).

A. Rebuilding

Not all monolithic application can be easily migrated to microservice architecture. Sometimes it is more economically beneficial to rebuilt application from scratch instead of refactoring it [21]. The following type of legacy applications are not recommended to refactor:

- Very old applications that are built using very old languages and databases.
- Applications what has a poor design.
- A application that are tightly coupled to the database.

B. Refactor

A different approach to legacy application modernization is to refactor everything with the goal of splitting legacy apps into microservices and connecting these microservices into one platform [21]. A different way to split legacy monolith to microservices showed in Table I and Table II. Each of them has benefits and drawbacks. Some of them are very general and could be used with any type of application, while other are more specific and will work only with some assumptions.

TABLE I. Migration Methods I

Method	Benefits	Drawbacks
Alessandra Levcovitz, Ricardo Terra and Marco Tulio Valente describes a technique to identify microservices on monolithic systems [8]. Proposed technique consists of the following steps: • Database decomposition – map database tables into subsystems. • Dependency Graph – create dependency graph between facades, business functions and database tables. • Based on dependency graph identify pairs of facades and database tables. Map subsystems and identified pairs of facades and database tables. • Identify candidates to be transformed on microservices. For each distinct pair obtained on the prior step, inspect the code of the facade and business functions that are on the path from façade to database table in the dependency graph. • Create API gateways to turn the migration to microservices transparent to clients. API gateway consists of an intermediate layer between client side and server-side application.	The proposed technique was able to identify and classify all subsystems, and create and analyse the dependency graph when evaluating and classifying only database tables into business subsystems, which demands access only to the source code and the database mode.	In some scenarios, additional effort might need to migrate the subsystem to a set of microservices: Subsystems that share the same database table. Microservice that represents an operation that is always in the middle of another operation. Business operations that involve more than one business subsystem on a transaction scope.
Rui Chen, Shanshan Li and Zheng Li proposed a top-down analysis approach and developed a dataflow-driven decomposition algorithm [28]. The three-step process is defined: 1. Engineers together with users conduct business requirement analysis and construct a purified while detailed dataflow diagram of the business logic. 2. The algorithm combines the same operations with the same type of output data into a virtual abstract dataflow. 3. The algorithm extracts individual modules of "operation and its output data" from the virtual abstract dataflow to represent the identified microservice candidates.	Dataflow driven mechanism guarantees the most fine-grained microservice candidates in terms of data operation within a business logic. Extraction can accept various text sources besides the webpage content, and it cares little about where its output data will go.	Identifying the same data operations requires expertise to some extent. Candidate microservices obtained from suggested decomposition mechanism could still need expert judgment before being developed in practice. Proposed decomposition mechanism has not been widely applied to large-scale projects.

Authorized licensed use limited to: Auckland University of Technology. Downloaded on August 21,2021 at 21:26:17 UTC from IEEE Xplore. Restrictions apply.

TABLE II. MIGRATION METHODS II

Method	Benefits	Drawbacks
Genc Mazlami, Jurgen Cito and Philipp Leitner presenting formal microservice extraction model to allow algorithmic recommendation of microservice candidates in a refactoring and migration scenario. The authors present a tool that supports structured service decomposition through graph cutting. The internal representation of the system to be decomposed is based on a catalog of 16 different coupling criteria that were abstracted from literature and industry know-how. Software engineering artifacts and documents such as domain models and use cases act as an input to generate the coupling values that build the graph representation [7].	The performance evaluation shows that, for the most part, proposed approach scales with respect to the size of the revision history (logical and contributor coupling). The quality evaluation shows that the proposed approach can reduce the microservice team size to a quarter of the monolith's team size or even lower.	One limitation is the fact that the extraction model is based on classes as the atomic unit of computation in the strategies and the graph. Using methods, procedures or functions as atomic units of extraction might potentially improve the granularity and precision of the code rearrangement and reorganization.
Holger Knoche and Wilhelm Hasslbring proposed a migration process to decompose an application into microservices [6]. Modernization process consists of five steps: 1. Defining an External Service Facade –defining an external service facade that captures the functionality required by the client systems in the form of well-defined service operations. 2. Adapting the Service Façade – implement an external service façade functionality. 3. Migrating Clients to the Service Façade – start using newly created external service façade interfaces. 4. Establishing Internal Service Facades – restructure applications internally. 5. Replacing the Service Implementations with Microservices – application replaced with microservices.	Establishing well-defined, platform independent interfaces based on the bounded contexts of the underlying domain. Improving the evolvability of the application. Reducing the number of entry points and preventing access to internals, moving noncustomer functionality into separate components, and eliminating redundant and obsolete parts of the application.	Certain parts of the application cannot be modernized using the presented approach. In particular, some user interfaces, which are built on highly proprietary technologies, lack the necessary means for service abstraction.
Zhamak Dehghani proposed very formal migration process from monolith to microservice architecture [28]. The suggested flow consists of these principles: • Minimize dependency back to the Monolith. • Split sticky capabilities early. • Decouple vertically and release the data early. • Decouple what is important to the business and changes frequently. • Decouple capability and not code. • Go macro first, then micro. • Migrate in atomic evolutionary steps.	Migrating process with this approach can be divided into small steps. It is possible to safely stop and restore.	Very long migration process. It is very formal way without any measurements.
Chen-Yuan Fan and Shang-Pin proposed a migration process based on SDLC, including all of the methods and tools required during design, development, and implementation [30].	Specialized and simple: Microservices are designed to handle problems in a single domain. Fault Tolerance: one microservices fault can't broke entire application. Automated: automation tools used for building, deployment, monitoring.	Complex environment settings: The configuration is not as simple as in a Monolithic architecture system, and many automation tools must be carefully set up to achieve the desired results. Using more resources: Microservices use multiple tools to achieve architectural flexibility, such as Service Discovery and API Gateway.

V. CONCLUSIONS

Each legacy monolithic application is unique. A business object, complexity, technology stack, team size, team skills and other, these are things which could very different in each case. There is no best way to migrate from monolith to microservices only success stories and good practice. Microservices is a relatively new architectural style, resulting in a lack of general guidelines for migrating monoliths towards microservices.

To choose migration methods and technics which best suits for an organization is very a hard task. The first question which always should be answered: refactor or rebuild? In most cases there is not so many resources and time to completely rebuild the solution.

Microservices architecture provides a lot of different advantages, but it also increases a solution complexity. To deal with increased complexity organization should prepare the specific infrastructure for microservices.

Each organization has its own reasons and goals, why migrate from monolith to microservices, was chosen. Migration plan should focus on these goals.

Authorized licensed use limited to: Auckland University of Technology. Downloaded on August 21,2021 at 21:26:17 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] M. Fowler and J. Lewis, "Microservices." [Online]. Available: http://martinfowler.com/articles/microservices.html

[2] Z. Dehghani, "How to break a Monolith into Microservices." [Online]. Available: https://martinfowler.com/articles/break-monolith-into-microservices.html

[3] O. Pozdniakova and D. Mažeika, "Systematic Literature Review of the Cloud-ready Software Architecture." Baltic J. Modern Computing, vol. 5, pp .124–135, March 2017

[4] O. Pozdniakova and D. Mažeika, "A cloud software isolation and cross-platform portability methods." 2017 Open Conference of Electrical, Electronic and Information Sciences (eStream), Vilnius, 2017, pp. 1–6.

[5] A. Balalaie, A. Heydarnoori and P. Jamshidi, "Microservices Architecture Enables DevOps: Migration to a Cloud-Native Architecture," in IEEE Software, vol. 33, no. 3, pp. 42–52, May-June 2016.

[6] H. Knoche and W. Hasselbring, "Using Microservices for Legacy Software Modernization," in IEEE Software, vol. 35, no. 3, pp. 44–49, May/June 2018.

[7] G. Mazlami, J. Cito and P. Leitner, "Extraction of Microservices from Monolithic Software Architectures," 2017 IEEE International Conference on Web Services (ICWS), Honolulu, HI, 2017, pp. 524–531.

[8] A. Levcovitz, R. Terra, M. T. Valente, "Towards a Technique for Extracting Microservices from Monolithic Enterprise Systems." 3rd Brazilian Workshop on Software Visualization, Evolution and Maintenance (VEM), p. 97–104, 2015

[9] Mulesoft, "https://www.mulesoft.com/resources/api/microservices-devops-better-together." [Online]. Available: https://www.mulesoft.com/resources/api/microservices-devops-better-together

[10] M. Andrawos, "Modern cloud native architecture: What you need to know about micro-services, containers and serverless." [Online]. Available: http://superuser.openstack.org/articles/modern-cloud-native-architecture-what-you-need-to-know-about-micro-services-containers-and-serverless/

[11] T. Nolle, "The right language for microservice orchestration and development." [Online]. Available: https://www.theserverside.com/feature/The-right-language-for-microservice-orchestration-and-development

[12] L. Atchison, "Microservice Architectures: What They Are and Why You Should Use Them." [Online]. Available: https://blog.newrelic.com/technology/microservices-what-they-are-why-to-use-them/

[13] B. Mayer and R. Weinreich, "An Approach to Extract the Architecture of Microservice-Based Software Systems," 2018 IEEE Symposium on Service-Oriented System Engineering (SOSE), Bamberg, 2018, pp. 21–30.

[14] M. Douglass, "Understanding Microservices: From Idea to Starting Line." [Online]. Available: https://medium.freecodecamp.org/microservices-from-idea-to-starting-line-ae5317a6ff02

[15] I. Azarny, "CI/CD for Containerized Microservices." [Online]. Available: https://dzone.com/articles/cicd-for-containerised-microservices

[16] H. Rosendahl, "Containers vs Virtual Machines (vms) – A Security Perspective." [Online]. Available: https://neuvector.com/container-security/containers-vs-virtual-machines-vms/

[17] A. Dave and L. Degioanni, "The Five Principles of Monitoring Microservices." [Online]. Available: https://thenewstack.io/five-principles-monitoring-microservices/

[18] A. Soroko, "Cloud Foundry Deployment Metrics That Matter Most." [Online]. Available: https://www.altoros.com/blog/cloud-foundry-deployment-metrics-that-matter-most/

[19] C. Melendez and D. McAllister, "Microservices Logging Best Practices." [Online]. Available: https://dzone.com/articles/microservices-logging-best-practices

[20] S. Anees, "How to Migrate to Microservices." [Online]. Available: https://blog.appdynamics.com/product/how-to-migrate-to-microservices/

[21] D. Linthicum, "From containers to microservices: Modernizing legacy applications." [Online]. Available: https://techbeacon.com/enterprise-it/containers-microservices-modernizing-legacy-applications

[22] N. Vennaro, "How to introduce microservices in a legacy environment." [Online]. Available: https://www.infoworld.com/article/3237175/how-to-introduce-microservices-in-a-legacy-environment.html

[23] S. Koltovich, "How to Modernize Legacy Applications for a Microservices-Based Deployment." [Online]. Available: https://thenewstack.io/modernize-legacy-applications-keep-update-re-write-needs-re-written/

[24] Carrasco, Andrés & van Bladel, Brent & Demeyer, Serge. "Migrating towards microservices: migration and architecture smells". IWoR 2018 Proceedings of the 2nd International Workshop on Refactoring, Montpellier, 2018, pp. 1–6.

[25] A. Furda, C. Fidge, O. Zimmermann, W. Kelly and A. Barros, "Migrating Enterprise Legacy Source Code to Microservices: On Multitenancy, Statefulness, and Data Consistency," in IEEE Software, vol. 35, no. 3, pp. 63–72, May/June 2018.

[26] M. Mishra, S. Kunde and M. Nambiar, "Cracking the Monolith: Challenges in Data Transitioning to Cloud Native Architectures." ECSA '18 Proceedings of the 12th European Conference on Software Architecture: Companion Proceedings, Madrid, 2018, pp. 1–4.

[27] R. Blanch, "Microservices: Strategies for Migration in a Brownfield Environment." [Online]. Available: https://medium.com/@rhettblanch_48135/microservices-strategies-for-migration-in-a-brownfield-environment-6c14335a8069

[28] R. Chen, S. Li and Z. Li, "From Monolith to Microservices: A Dataflow-Driven Approach," 2017 24th Asia-Pacific Software Engineering Conference (APSEC), Nanjing, 2017, pp. 466–475.

[29] M. Mazzara, N. Dragoni, A. Bucchiarone, A. Giaretta, S. T. Larsen and S. Dustdar, "Microservices: Migration of a Mission Critical System," in IEEE Transactions on Services Computing.

[30] C. Fan and S. Ma, "Migrating Monolithic Mobile Application to Microservice Architecture: An Experiment Report," 2017 IEEE International Conference on AI & Mobile Services (AIMS), Honolulu, HI, 2017, pp. 109–112.

Authorized licensed use limited to: Auckland University of Technology. Downloaded on August 21,2021 at 21:26:17 UTC from IEEE Xplore. Restrictions apply.